VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

AYUSH S SHETTY(1BM24CS064)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by AYUSH S SHETTY**(1BM24CS064)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Prof. Syed Akram
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write a program to obtain the Topological ordering of vertices in a given digraph.

C Code:

```
#include <stdio.h>
#define MAX 10
int main() {
   int n, i, j, graph[MAX][MAX], indegree[MAX] = {0};
   int queue[MAX], front = 0, rear = -1, topo[MAX], k = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("Enter adjacency matrix:\n");
   for (i = 0; i < n; i++) {
      for (j = 0; j < n; j++) {
         scanf("%d", &graph[i][j]);
         if (graph[i][j] == 1) indegree[j]++;
      }
   }
   for (i = 0; i < n; i++) if (indegree[i] == 0) queue[++rear] = i;
   while (front <= rear) {
      int v = queue[front++]; topo[k++] = v;
      for (j = 0; j < n; j++) {
         if (graph[v][j] == 1) {
            indegree[j]--;
            if (indegree[j] == 0) queue[++rear] = j;
         }
      }
   }
   printf("Topological Order: ");
   for (i = 0; i < k; i++) printf("%d ", topo[i]);
   return 0;
}
```

OUTPUT:

```
Enter number of vertices: 3
Enter adjacency matrix:
0 1 0
0 0 1
0 0 0
Topological Order: 0 1 2
Process returned 0 (0x0)   execution time : 28.434 s
Press any key to continue.
```

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

C Code:

```
#include <stdio.h>

int getMobile(int a[], int dir[], int n)
{
   int mobile = 0, mobile_pos = -1;

   for(int i = 0; i < n; i++)
   {
      if(dir[i] == -1 && i != 0 && a[i] > a[i-1])   // Left
      {
         if(a[i] > mobile)
         {
            mobile = a[i];
            mobile_pos = i;
         }
      }

      if(dir[i] == 1 && i != n-1 && a[i] > a[i+1])  // Right
      {
         if(a[i] > mobile)
         {
            mobile = a[i];
            mobile_pos = i;
         }
      }
   }

   return mobile_pos;
}

void printPermutation(int a[], int n)
{
   for(int i = 0; i < n; i++)
      printf("%d ", a[i]);
   printf("\n");
}

int main()
{
   int n;

   printf("Enter n: ");
   scanf("%d", &n);
```

```
int a[n], dir[n];

// Initial permutation
for(int i = 0; i < n; i++)
{
   a[i] = i + 1;
   dir[i] = -1;   // -1 = Left, 1 = Right
}

printf("Permutations:\n");
printPermutation(a, n);

while(1)
{
   int pos = getMobile(a, dir, n);

   if(pos == -1)
      break;

   int mobile = a[pos];

   // Swap mobile element
   if(dir[pos] == -1)
   {
      int temp = a[pos];
      a[pos] = a[pos-1];
      a[pos-1] = temp;

      temp = dir[pos];
      dir[pos] = dir[pos-1];
      dir[pos-1] = temp;

      pos--;
   }
   else
   {
      int temp = a[pos];
      a[pos] = a[pos+1];
      a[pos+1] = temp;

      temp = dir[pos];
      dir[pos] = dir[pos+1];
      dir[pos+1] = temp;

      pos++;
   }

   // Reverse directions of elements larger than mobile
   for(int i = 0; i < n; i++)
```

```
        {
            if(a[i] > mobile)
                dir[i] = -dir[i];
        }

        printPermutation(a, n);
    }

    return 0;
}
```

OUTPUT:

```
Enter the number of elements (n): 3

Starting Permutation:
1<- 2<- 3<-
1<- 3<- 2<-
3<- 1<- 2<-
3-> 2<- 1<-
2<- 3-> 1<-
2<- 1<- 3->

Process returned 0 (0x0)   execution time : 48.162 s
Press any key to continue.
```

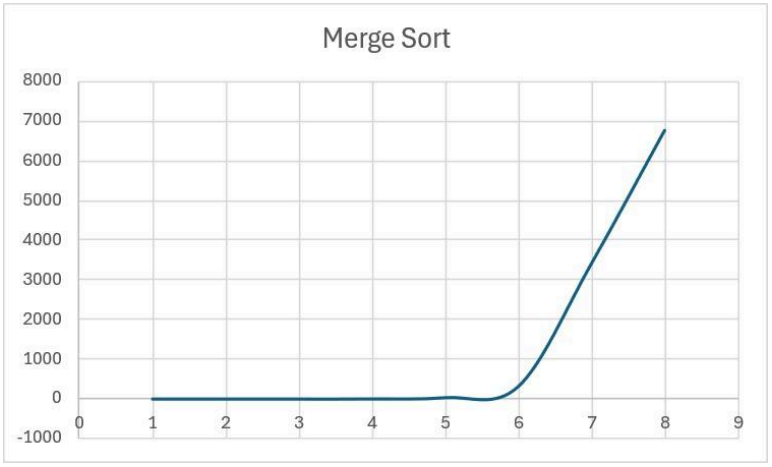
Lab Program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

C Code:

```
#include <stdio.h>

void merge(int arr[], int l, int m, int r)
{
   int i, j, k;
   int n1 = m - l + 1;
   int n2 = r - m;

   int L[n1], R[n2];

   for (i = 0; i < n1; i++)
      L[i] = arr[l + i];
   for (j = 0; j < n2; j++)
      R[j] = arr[m + 1 + j];
   i = 0;
   j = 0;
   k = l;


   while (i < n1 && j < n2)
   {
      if (L[i] <= R[j])
      {
         arr[k] = L[i];
         i++;
      }
      else
      {
         arr[k] = R[j];
         j++;
      }
      k++;
   }
   while (i < n1)
   {
      arr[k] = L[i];
      i++;
      k++;
   }
   while (j < n2)
   {
      arr[k] = R[j];
```

```
        j++;
        k++;
    }
}

void mergesort(int arr[], int l, int r)
{
    if (l < r)
    {
        int m = l + (r - l) / 2;
        mergesort(arr, l, m);
        mergesort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main()
{
    int n, i;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter the elements: ");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    mergesort(a, 0, n - 1);
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
    return 0;
}
```



OUTPUT:

```
Enter the number of elements: 5
Enter the elements: 32 34 54 21 32
Sorted array: 21 32 32 34 54

Process returned 0 (0x0)   execution time : 14.935 s
Press any key to continue.
```

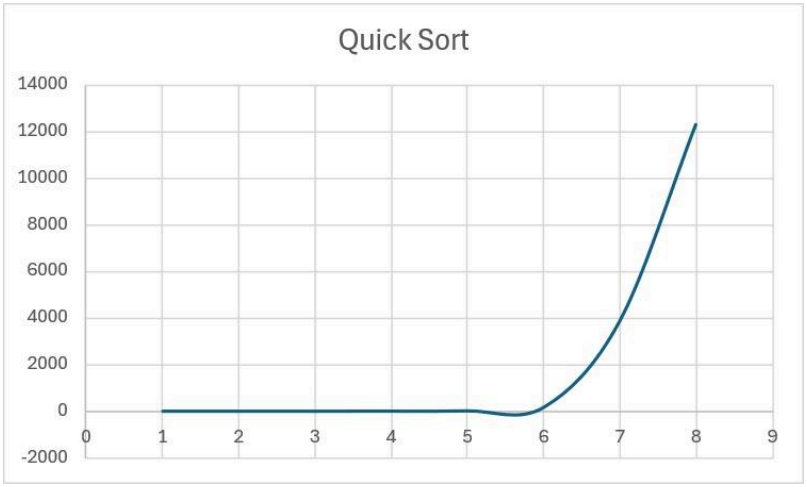
Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

C Code:

```
#include <stdio.h>

int partition(int a[], int low, int high) {
   int key = a[low];
   int i = low + 1;
   int j = high;
   int temp;

   while (i <= j) {
      while (i <= high && a[i] <= key) {
         i++;
      }
      while (a[j] > key) {
         j--;
      }
      if (i < j) {
         temp = a[i];
         a[i] = a[j];
         a[j] = temp;
      }
   }

   temp = a[low];
   a[low] = a[j];
   a[j] = temp;
   return j;
}
```

```
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

int main() {
    int n, i;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter the elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    quickSort(a, 0, n - 1);

    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    return 0;
}
```



Output:

```
Enter the number of elements: 7
Enter the elements: 11 23 43 56 76 54 67 87 98
Sorted array: 11 23 43 54 56 67 76

Process returned 0 (0x0)   execution time : 20.961 s
Press any key to continue.
```

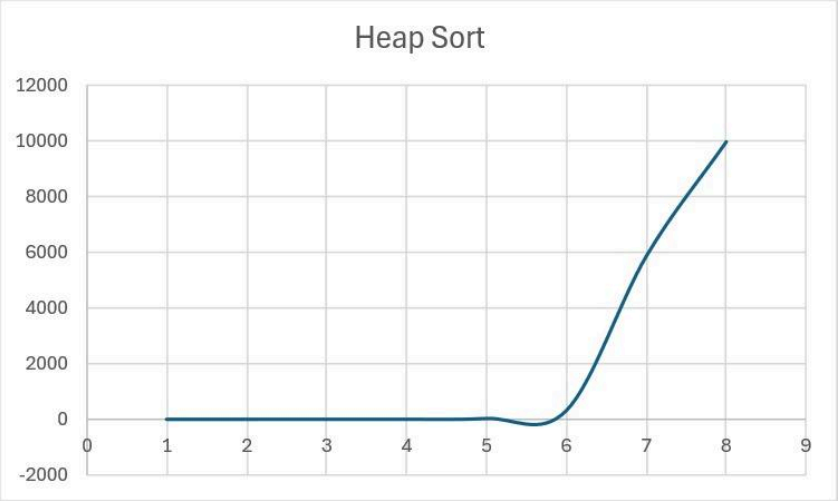
Lab Program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

C Code:

```
#include <stdio.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int l = 2 * i + 1;
    int r = 2 * i + 2;

    if (l < n && arr[l] > arr[largest])
        largest = l;

    if (r < n && arr[r] > arr[largest])
        largest = r;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}
```

```
int main() {
    int n, i;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter the elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    heapSort(a, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    return 0;
}
```



Output:

```
Enter the number of elements: 8
Enter the elements: 9 4 3 8 10 2 5 7
Sorted array: 2 3 4 5 7 8 9 10

Process returned 0 (0x0)   execution time : 35.063 s
Press any key to continue.
```

Lab Program 6:

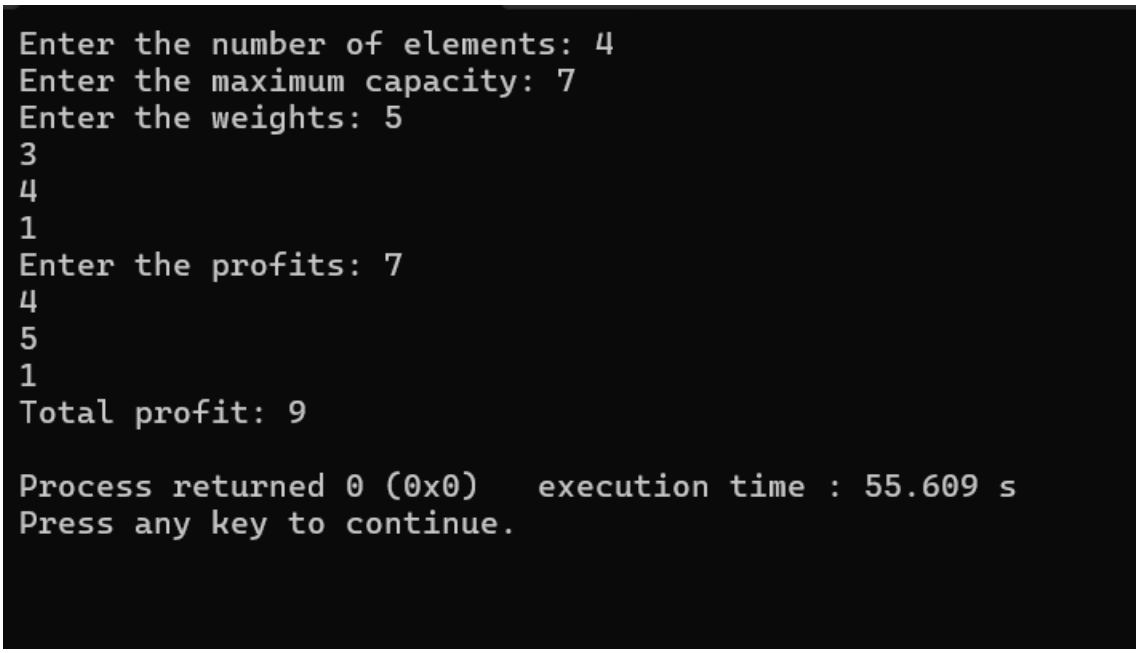
Implement 0/1 Knapsack problem using dynamic programming.

C Code:

```
#include <stdio.h>

int main()
{
   int n, w[50], p[50], m, a[50][50];
   int i, j;
   printf("Enter the number of elements: ");
   scanf("%d", &n);
   printf("Enter the maximum capacity: ");
   scanf("%d", &m);
   printf("Enter the weights: ");
   for (i = 1; i <= n; i++)
   {
      scanf("%d", &w[i]);
   }
   printf("Enter the profits: ");
   for (i = 1; i <= n; i++)
   {
      scanf("%d", &p[i]);
   }
   for (i = 0; i <= n; i++)
      a[i][0] = 0;
   for (j = 0; j <= m; j++)
      a[0][j] = 0;
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= m; j++)
      {
         if (j < w[i])
         {
            a[i][j] = a[i - 1][j];
         }
         else
         {
            if (a[i - 1][j] > (p[i] + a[i - 1][j - w[i]]))
            {
               a[i][j] = a[i - 1][j];
            }
            else
            {
               a[i][j] = p[i] + a[i - 1][j - w[i]];
            }
         }
```

```
        }
    }
    printf("Total profit: %d\n", a[n][m]);
    return 0;
}
```

Output:



Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

C Code:

```
#include <stdio.h>

void Floyds(int n, int cost[50][50], int A[50][50])
{
   int i, j, k;

   for(i = 1; i <= n; i++)
      for(j = 1; j <= n; j++)
         A[i][j] = cost[i][j];

   for(k = 1; k <= n; k++)
      for(i = 1; i <= n; i++)
         for(j = 1; j <= n; j++)
            if(A[i][k] + A[k][j] < A[i][j])
               A[i][j] = A[i][k] + A[k][j];
}

int main() {
   int n, i, j;
   int cost[50][50], A[50][50];
   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("Enter the cost matrix (use 999 for infinity):\n");
   for(i = 1; i <= n; i++) {
      for(j = 1; j <= n; j++) {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0 && i != j) {
            cost[i][j] = 999;
         }
      }
   }
   Floyds(n, cost, A);
   printf("\nThe all-pairs shortest path matrix is:\n");
   for(i = 1; i <= n; i++) {
      for(j = 1; j <= n; j++) {
         if (A[i][j] == 999) printf("INF ");
         else printf("%d  ", A[i][j]);
      }
      printf("\n");
   }

   return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter the cost matrix (use 999 for infinity):
0 999 6 1
4 0 20 10
999 3 0 12
6 999 999 0

The all-pairs shortest path matrix is:
0  9  6  1
4  0  10  5
7  3  0  8
6  15  12  0

Process returned 0 (0x0)   execution time : 54.621 s
Press any key to continue.
```

Lab Program 8a:

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

C Code:

```
#include <stdio.h>


int main()
{
   int n, cost[10][10];
   int visited[10] = {0};
   int min, u, v, edges = 0;
   int mincost = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter cost matrix:\n");
   for(int i = 0; i < n; i++)
   {
      for(int j = 0; j < n; j++)
      {
         scanf("%d", &cost[i][j]);

         if(cost[i][j] == 0)
            cost[i][j] = 999;   // No edge
      }
   }
```

```
visited[0] = 1;  // Start from vertex 0

printf("\nEdges in MST:\n");

while(edges < n - 1)
{
   min = 999;

   for(int i = 0; i < n; i++)
   {
      if(visited[i])
      {
         for(int j = 0; j < n; j++)
         {
            if(!visited[j] && cost[i][j] < min)
            {
               min = cost[i][j];
               u = i;
               v = j;
            }
         }
      }
   }

   printf("%d -> %d : %d\n", u, v, min);
```

```
        mincost += min;
        visited[v] = 1;
        edges++;
    }

    printf("\nTotal minimum Cost : %d\n", mincost);

    return 0;
}
```

Output:

```
Enter no of vertices: 4
Enter Adjacency matrix (use 999 for no edge):
0 2 999 6
2 0 3 8
999 3 0 0
6 8 0 0

Edge    Weight
0 - 1    2
1 - 2    3
0 - 3    6

Total minimum Cost: 11
```

Lab Program 8 b:

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

C Code:

```
#include <stdio.h>

int parent[10];

int find(int v)
{
   while(parent[v] != 0)
      v = parent[v];
   return v;
}

void unionSet(int u, int v)
{
   parent[v] = u;
}

int main()
{
   int n, cost[10][10];
   int mincost = 0;
   int edges = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter cost matrix:\n");
   for(int i = 0; i < n; i++)
   {
      for(int j = 0; j < n; j++)
      {
         scanf("%d", &cost[i][j]);

         if(cost[i][j] == 0)
            cost[i][j] = 999;
      }
   }

   printf("\nEdges in MST:\n");

   while(edges < n - 1)
   {
      int min = 999;
      int u = -1, v = -1;
```

```
        // Find minimum edge
        for(int i = 0; i < n; i++)
        {
            for(int j = 0; j < n; j++)
            {
                if(cost[i][j] < min)
                {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }

        int p = find(u);
        int q = find(v);

        // If no cycle
        if(p != q)
        {
            printf("%d -> %d : %d\n", u, v, min);
            mincost += min;
            edges++;
            unionSet(p, q);
        }

        cost[u][v] = cost[v][u] = 999;
    }

    printf("\nMinimum Cost = %d\n", mincost);

    return 0;
}
```

Output:

```
Enter the number of edges: 5
Enter src, dest and cost for following
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

For the total cost is: 19

Process returned 0 (0x0)   execution time : 39.230 s
Press any key to continue.
```

Lab Program 9:

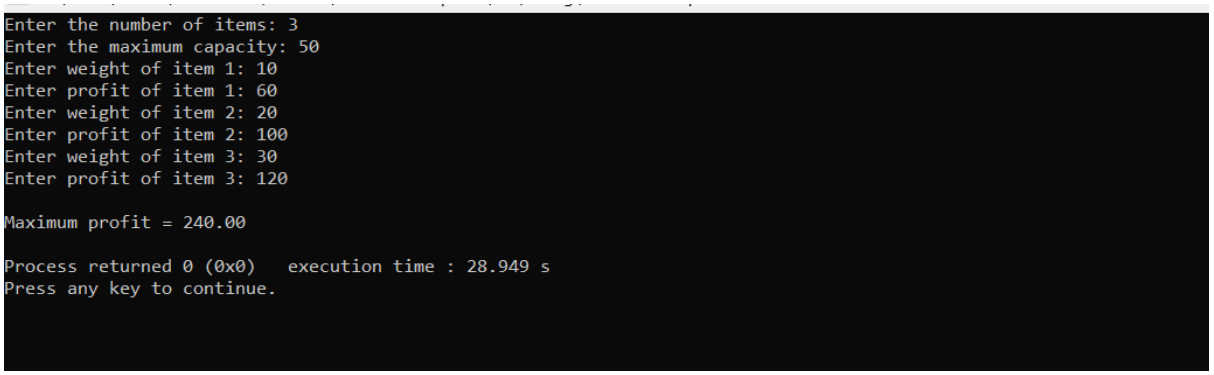
Implement fractional knapsack problem using Greedy technique.

C Code:

```
#include<stdio.h>
struct item{
   int p;
   int w;
   float r;
   float x;
};
int main(){
   int n, m;
   printf("Enter the number of items: ");
   scanf("%d", &n);
   printf("Enter the maximum capacity: ");
   scanf("%d", &m);
   struct item items[20], temp;
   for(int i=0; i<n; i++){
      printf("Enter weight of item %d: ", i+1);
      scanf("%d", &items[i].w);
      printf("Enter profit of item %d: ", i+1);
      scanf("%d", &items[i].p);
      items[i].x = 0;
      items[i].r = (float)items[i].p / items[i].w;
   }
   for(int i=0; i<n; i++){
      for(int j=i+1; j<n; j++){
         if(items[j].r > items[i].r){
            temp = items[i];
            items[i] = items[j];
            items[j] = temp;
         }
      }
   }
   float remain = m, maxprofit = 0;
   for(int i=0; i<n; i++){
      if(items[i].w <= remain){
         items[i].x = 1;
         remain -= items[i].w;
         maxprofit += items[i].p;
      } else {
         items[i].x = remain / items[i].w;
         maxprofit += items[i].p * items[i].x;
         break;
         }
   }
```

```
    printf("\nMaximum profit = %.2f\n", maxprofit);
}
```

Output:



Lab Program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

C Code

```
#include <stdio.h>

int main()
{
   int n, cost[10][10];
   int dist[10], visited[10];
   int source;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter cost matrix:\n");
   for(int i = 0; i < n; i++)
   {
      for(int j = 0; j < n; j++)
      {
         scanf("%d", &cost[i][j]);

         if(i != j && cost[i][j] == 0)
            cost[i][j] = 999;
      }
   }

   printf("Enter source vertex: ");
   scanf("%d", &source);

   for(int i = 0; i < n; i++)
   {
      dist[i] = cost[source][i];
      visited[i] = 0;
   }

   dist[source] = 0;
   visited[source] = 1;

   for(int count = 1; count < n; count++)
   {
      int min = 999, u = -1;

      // Find nearest unvisited vertex
      for(int i = 0; i < n; i++)
      {
         if(!visited[i] && dist[i] < min)
```

```
            {
                min = dist[i];
                u = i;
            }
        }

        visited[u] = 1;

        // Relax adjacent vertices
        for(int v = 0; v < n; v++)
        {
            if(!visited[v] &&
               dist[u] + cost[u][v] < dist[v])
            {
                dist[v] = dist[u] + cost[u][v];
            }
        }
    }

    printf("\nShortest distances from vertex %d:\n", source);

    for(int i = 0; i < n; i++)
    {
        printf("%d -> %d = %d\n", source, i, dist[i]);
    }

    return 0;
}
```

Output:

```
Enter number of vertices
4
Enter the cost matrix
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0
Enter source vertex
0
Shortest distances from vertex 0:
to vertex 1 = 5
to vertex 2 = 3
to vertex 3 = 0
to vertex 4 = 11
```

Lab Program 11:

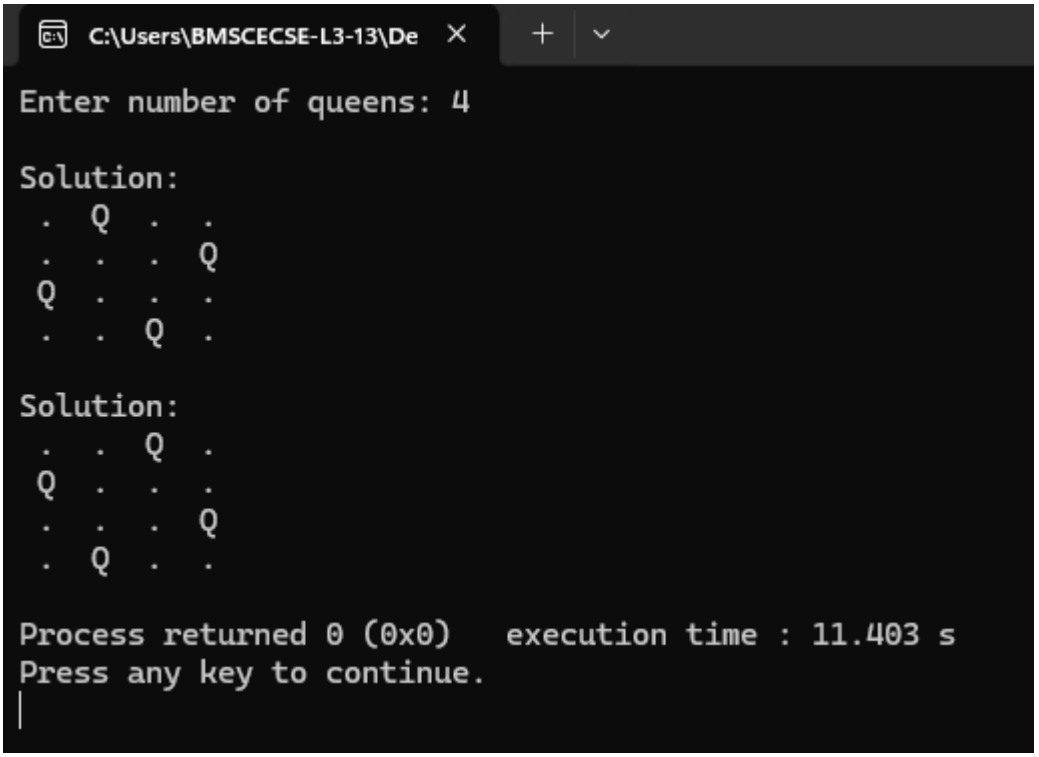
Implement “N-Queens Problem” using Backtracking.

C Code:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 20
int x[MAX];
int PLACE(int k, int i)
{
   int j;
   for (j = 1; j <= k - 1; j++)
   {
      if ((x[j] == i) || (abs(x[j] - i) == abs(j - k)))
         return 0;
   }
   return 1;
}

void printSolution(int n)
{
   int i, j;
   printf("\nSolution:\n");
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= n; j++)
      {
         if (x[i] == j)
            printf(" Q ");
         else
            printf(" . ");
      }
      printf("\n");
   }
}

void NQueens(int n)
{
   int k = 1;
   x[k] = 0;
   while (k != 0)
   {
      x[k] = x[k] + 1;
      while ((x[k] <= n) && !PLACE(k, x[k]))
      {
         x[k] = x[k] + 1;
      }
      if (x[k] <= n)
```

```
        {
            if (k == n)
            {
                printSolution(n);
            }
            else
            {
                k++;
                x[k] = 0;
            }
        }
        else
        {
            k--;
        }
    }
}

int main()
{
    int n;
    printf("Enter number of queens: ");
    scanf("%d", &n);
    NQueens(n);
}
```

Output:

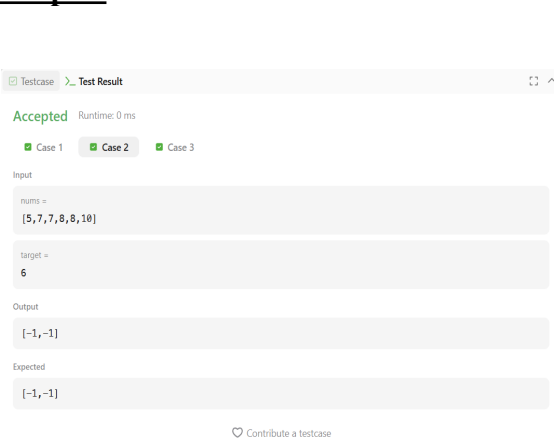


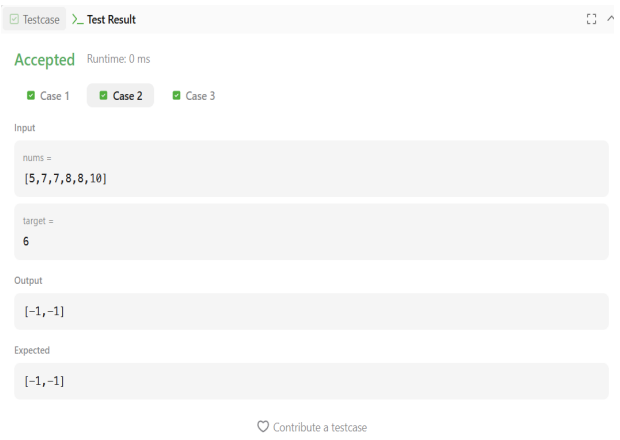
LEETCODE 34. Find First and Last Position of Element in Sorted Array

Program:

```
#include <stdlib.h>

int* searchRange(int* nums, int numsSize, int target, int* returnSize) {
    *returnSize = 2;

    int* result = (int*)malloc(2 * sizeof(int));
    result[0] = -1;
    result[1] = -1;

    for (int i = 0; i < numsSize; i++) {
        if (nums[i] == target) {
            result[0] = i;
            break;
        }
    }
    for (int i = numsSize - 1; i >= 0; i--) {
        if (nums[i] == target) {
            result[1] = i;
            break;
        }
    }
    return result;
}
```

Output:



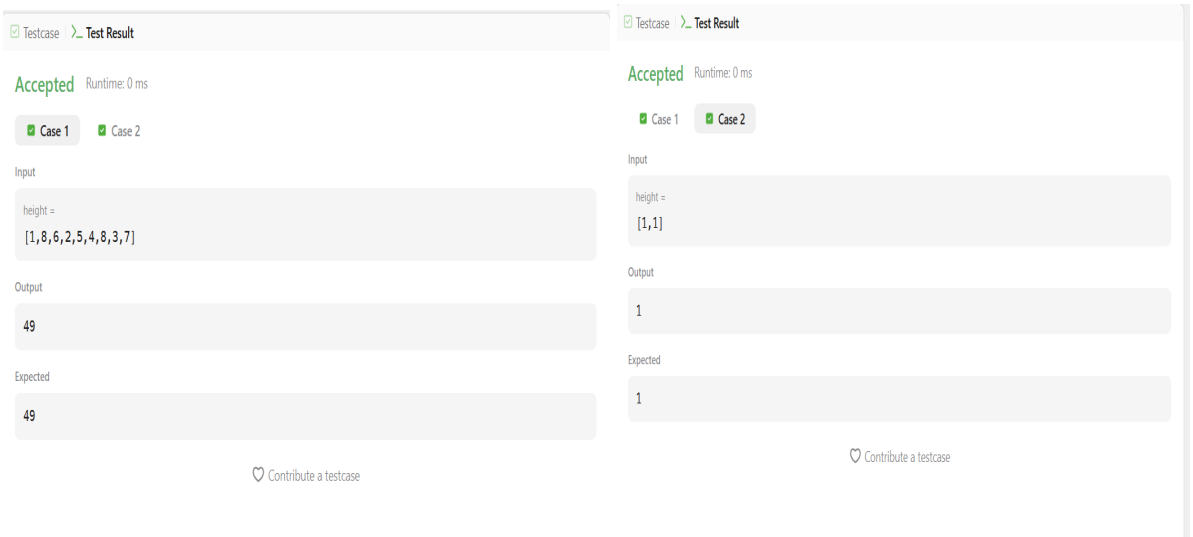


LEETCODE 11. Container With Most Water

Program:

```
int maxArea(int* height, int heightSize) {
    int i=0;
    int j=heightSize-1;
    int res=0;
    while(i<j){
        int minH=height[i]<height[j]?height[i]:height[j];
        int area=(j-i)*minH;
        if(area>res){
            res=area;
        }
        if(height[i]<height[j]){
            i++;
        }
        else{
            j--;
        }
    }return res;
}
```

Output:

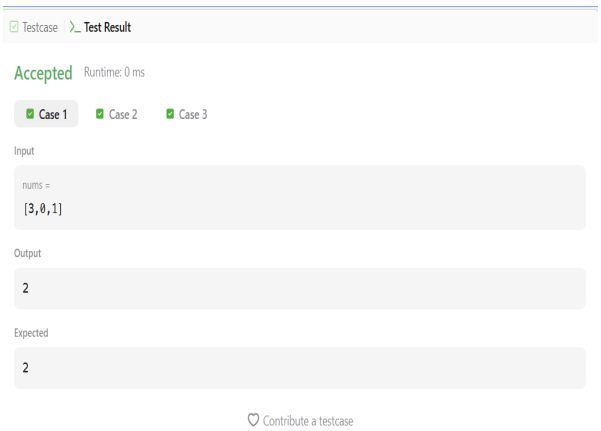


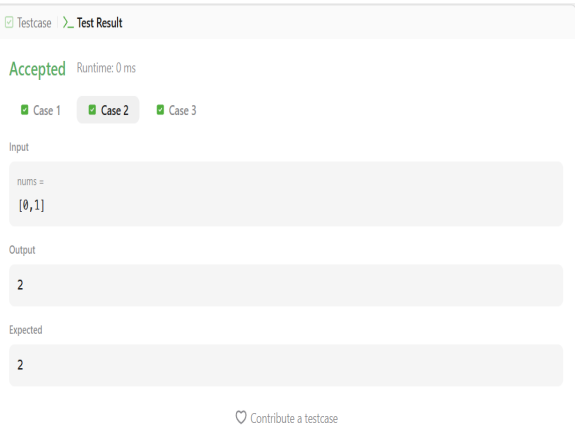
LEETCODE 268. Missing Number

Program:

```
int missingNumber(int* nums, int numsSize) {
    int i, j;

    for(i = 0; i < numsSize - 1; i++) {
        for(j = 0; j < numsSize - i - 1; j++) {
            if(nums[j] > nums[j + 1]) {
                int temp = nums[j];
                nums[j] = nums[j + 1];
                nums[j + 1] = temp;
            }
        }
    }

    for(i = 0; i < numsSize; i++) {
        if(nums[i] != i) {
            return i;
    }

    return numsSize;
}
```

Output:



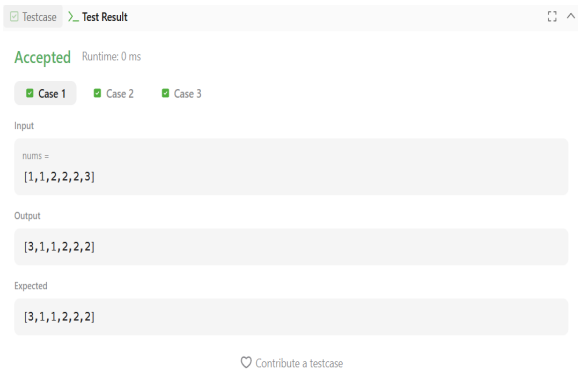


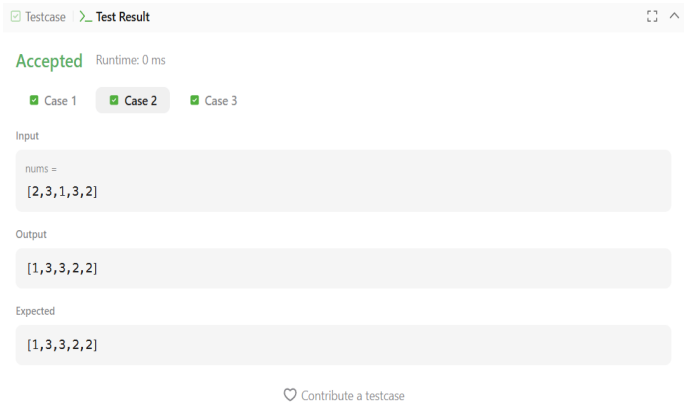
LEETCODE 1636. Sort Array by Increasing Frequency

Program:

```
int* frequencySort(int* nums, int numsSize, int* returnSize) {
    *returnSize = numsSize;
    int i, j, k;
    for (i = 0; i < numsSize - 1; i++) {
        for (j = 0; j < numsSize - i - 1; j++) {
            int f1 = 0, f2 = 0;
            for (k = 0; k < numsSize; k++) {
                if (nums[k] == nums[j])   f1++;
                if (nums[k] == nums[j+1]) f2++;
            }
            if (f1 > f2 || (f1 == f2 && nums[j] < nums[j+1])) {
                int temp = nums[j];
                nums[j] = nums[j+1];
                nums[j+1] = temp;
            }
        }
    }
    return nums;
}
```

Output:



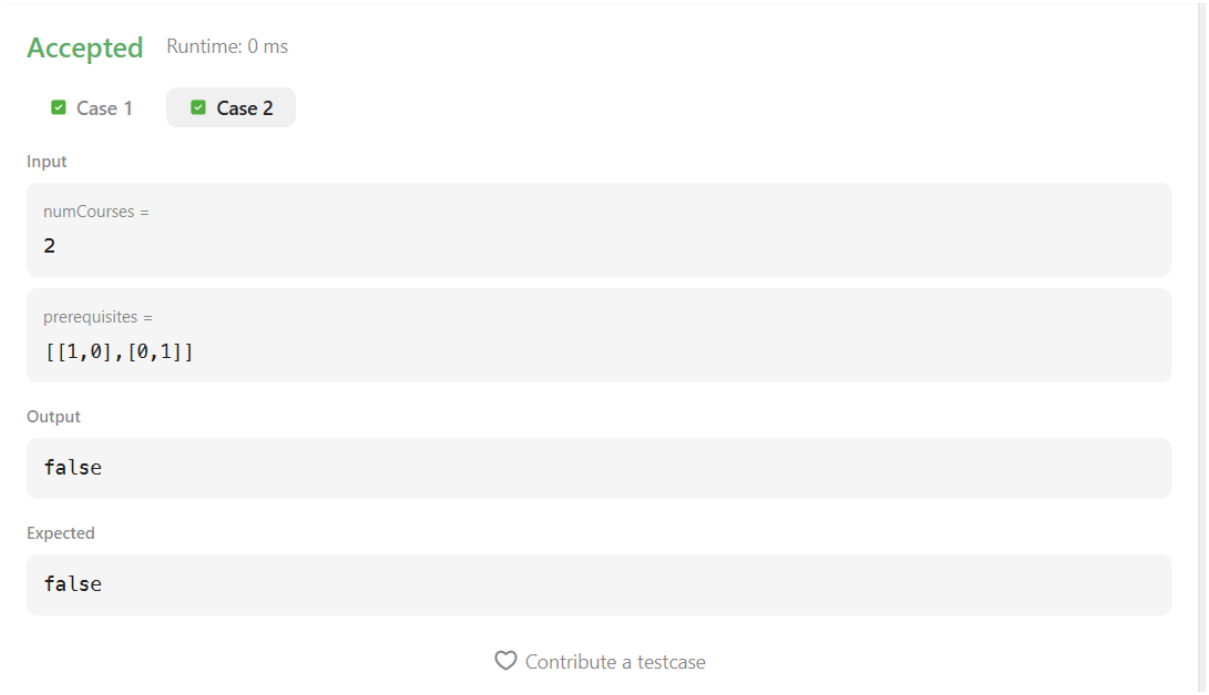


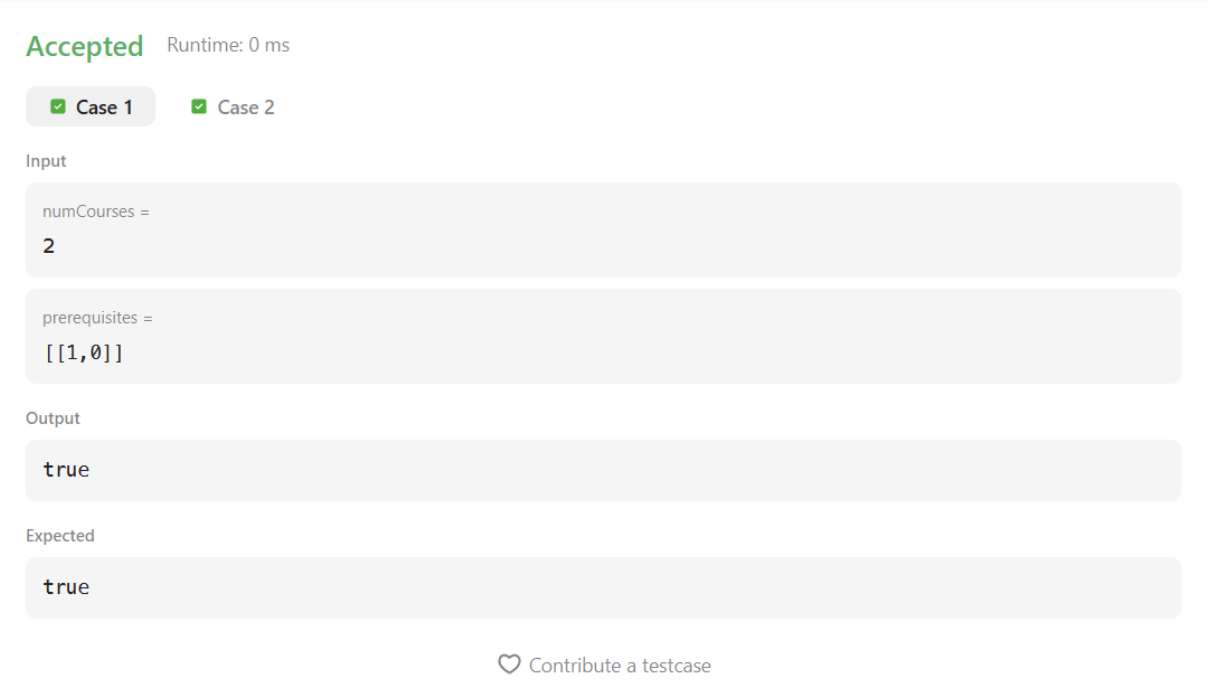
LEETCODE 207. Course Schedule

Program:

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int *indegree = calloc(numCourses, sizeof(int));
    int *outDegree = calloc(numCourses, sizeof(int));
    for (int i = 0; i < prerequisitesSize; i++) {
        outDegree[prerequisites[i][1]]++;
        indegree[prerequisites[i][0]]++;
    }
    int **graph = malloc(numCourses * sizeof(int *));
    for (int i = 0; i < numCourses; i++) {
        graph[i] = malloc(outDegree[i] * sizeof(int));
        outDegree[i] = 0;
    }
    for (int i = 0; i < prerequisitesSize; i++) {
        int u = prerequisites[i][1];
        int v = prerequisites[i][0];
        graph[u][outDegree[u]++] = v;
    }
    int *queue = malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }
    int count = 0;
    while (front < rear) {
        int u = queue[front++];
        count++;
        for (int i = 0; i < outDegree[u]; i++) {
            int v = graph[u][i];
            if (--indegree[v] == 0) {
                queue[rear++] = v;
            }
        }
    }
    for (int i = 0; i < numCourses; i++) {
        free(graph[i]);
    }
    free(graph);
    free(indegree);
    free(outDegree);
    free(queue);
    return count == numCourses;
}
```

Output:


Accepted
Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2
prerequisites =
[[1,0],[0,1]]
Output
false
Expected
false
Contribute a testcase

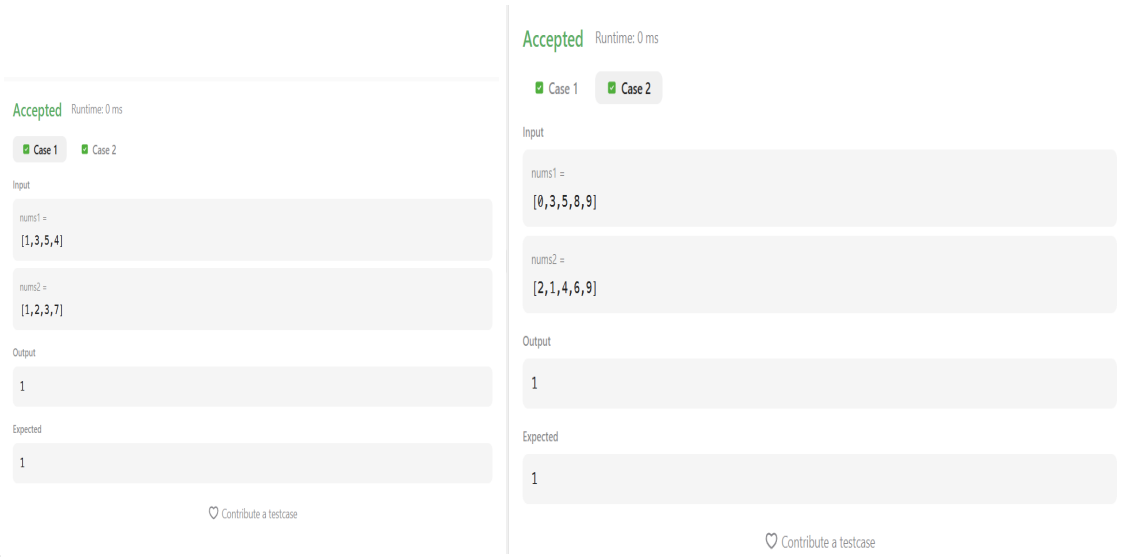

Accepted
Runtime: 0 ms
Case 1
Case 2
Input
numCourses =
2
prerequisites =
[[1,0]]
Output
true
Expected
true
Contribute a testcase

LEETCODE 801. Minimum Swaps To Make Sequences Increasing

Program:

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int keep = 0;
    int swap = 1;
    for (int i = 1; i < nums1Size; i++) {
        int nextKeep = nums1Size + 1;
        int nextSwap = nums1Size + 1;
        if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1]) {
            nextKeep = keep;
            nextSwap = swap + 1;
        }
        if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
            if (swap < nextKeep)
                nextKeep = swap;
            if (keep + 1 < nextSwap)
                nextSwap = keep + 1;
        }
        keep = nextKeep;
        swap = nextSwap;
    }
    return keep < swap ? keep : swap;
}
```

Output:



-

LEETCODE 287. Find the Duplicate Number

Program:

```
int findDuplicate(int* nums, int numsSize) {

    int slow = nums[0];
    int fast = nums[0];


    do{
        slow = nums[slow];
        fast = nums[nums[fast]];
    }while (slow !=fast);

    slow = nums[0];
    while (slow !=fast){
        slow = nums[slow];
        fast = nums[fast];
    }

     return slow;
}
```

Output: